Lab W14: The Weather Station Network

CIS 3300, Intermediate Object-Oriented Programming

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

1 The Story

The Prairie Ridge Weather Network runs five weather stations. Every evening, each station sends one line to the network's office with the day's high temperature, low temperature, and precipitation. The office collects the lines in one file and, at the end of each period, writes a report.

The trouble is that the lines are not always right. A sensor sends a temperature of 158 degrees. A technician types the letter O instead of a zero. A station that is not part of the network sends a reading. Lines arrive late, so they are not in order. Right now one bad line stops the office's program before it writes anything. You will turn that program into one that reads every good line, skips and records every bad one, and writes the report.

This lab is guided. It tells you which classes and methods to write, and it is divided into three parts: Part 1 covers exceptions and logging; Part 2 covers the `Comparable` interface, generics, and collections; and Part 3 covers lambdas and streams. Work through the tasks in order, because each one builds on the one before it. The lab is not due until Week 14, but you should finish at least one part each week, so that you practice each topic while it is fresh from lecture.

2 Project Setup

1. Create a new Java project named `labw14` with two packages: `weather`, for the classes that represent and analyze the data, and `app`, for the program with `main`.
2. Copy the provided `Station.java` into the `weather` package, and use it as it is. Copy the provided starter program, `WeatherReport.java`, into the `app` package; you will change it as you work.
3. Create a `files` folder in the project's root folder, and place the four provided data files and the two sample files in it. Your program writes its output files to the same folder.

When you finish, your project should look like Figure 1. If Eclipse also created a `module-info.java` file, you may keep it.

```
labw14
  src
    app
      WeatherReport.java            the starter, provided; you change it
    weather
      MinMax.java                   you write it
      Reading.java                  you write it
      ReadingParseException.java    you write it
      Station.java                  provided; use it as it is
      WeatherData.java              you write it
      WeatherStats.java             you write it
  files
    stations.csv                    provided
    readings.csv                    provided
    readings2.csv                   provided
    readings_clean.csv              provided
    sample_weather_report.txt       provided
    sample_errors.txt               provided
    weather_report.txt              created by your program
    errors.txt                      created by your program
```

Figure 1: The finished project's layout in Eclipse, with the packages, classes, and files, and where each one comes from.

3 The Files

All the data files are comma-separated text files whose first line is a header. Read each line as text and split it on commas; the files have no quoted fields, so you do not need a CSV library. When this lab gives a line number, it counts the header as line 1, so the numbers match what a text editor shows.

`stations.csv` lists the five stations: an ID, a name, a county, and an elevation in feet. This file has no errors.

`readings.csv` has one reading per line, in this form:

```
station,date,high,low,precipitation
ASH,2026-07-01,87.3,64.3,0.00
```

Table 1 lists what the network accepts. A line is *bad* if it does not have exactly five fields, or if any field is not accepted. Every other line is good.

Table 1: The values the network accepts in each reading.

Field	Accepted values
`station`	The ID of a station in `stations.csv`, exactly as written there
`date`	A real date written as year, month, and day, such as `2026-07-01`
`high`, `low`	A number from −80 to 140 degrees Fahrenheit. The high may not be below the low.
`precipitation`	A number of inches, 0 or more

`readings_clean.csv` has fifteen good readings, three days from every station, and no bad lines. **`readings2.csv`** is a second test file, described in Section 8.

4 The Starter Program

The provided `WeatherReport.java` reads `readings_clean.csv`, splits each line on commas, and prints each reading. Its `main` method is declared with `throws IOException`. Before you change anything, run it three ways:

1. **As it is.** It prints the fifteen readings, beginning:

```
ASH  2026-07-01   87.3   64.3  0.00
BEL  2026-07-01   95.5   75.6  0.00
```

2. **On the real file.** Change the file name to `readings.csv`. The program stops at the very first bad line, line 2, and never reads the rest:

```
Exception in thread "main" java.time.format.DateTimeParseException:
Text '2026-06-31' could not be parsed: Invalid date 'JUNE 31'
```

   The message is one line, broken here to fit the page.

3. **With a missing file.** Change the file name to one that does not exist, such as `reading_clean.csv`. The program prints a long stack trace, which begins:

```
Exception in thread "main" java.nio.file.NoSuchFileException:
files/reading_clean.csv
```

   The message is one line, broken here to fit the page.

Change the file name back to `readings_clean.csv`. The tasks fix both problems.

5 Part 1: Bad Data (Week 12)

5.1 Task 1: Catch the file problem

Remove `throws IOException` from `main`, and catch the exception instead. A missing file must print one line that names the file, and any other `IOException` must print one line that says what went wrong. Neither may show a stack trace. A missing file throws `NoSuchFileException`, a subclass of `IOException` whose `getFile()` method returns the file's name.

5.2 Task 2: The `Reading` record

In the `weather` package, write a *record* named `Reading`, declared with the keyword `record` rather than `class`, with these components, in this order: `String stationId`, `LocalDate date`, `double high`, `double low`, and `double precipitation`. A record is immutable: once a reading is created, its values can never change. That is why checking them once, in the constructor, is enough.

Give it a compact constructor that refuses any value Table 1 does not accept, so that no bad reading can ever exist. Throw `IllegalArgumentException` with a message that names the problem and the bad value, such as `high 68.2 is below low 92.5`. A `null` station ID or date must throw `NullPointerException`. Checking that the station exists is not this record's job, because a reading does not know which stations the network has.

5.3 Task 3: Your own exception class

Write a *checked* exception named `ReadingParseException` that represents one bad line in a readings file. Give it:

- a constructor that takes the line number and a message, and one that also takes a cause, each producing a message that starts with `line` and the line number, such as `line 19: high is not a number: "90.2"`;
- a method `int getLineNumber()`;
- a `serialVersionUID`.

5.4 Task 4: Turning one line into a reading

Write a class named `WeatherData`, and in it a static method that turns one line into a reading:

```
public static Reading parseReading(int lineNumber, String line,
        Map<String, Station> stations) throws ReadingParseException
```

Every problem with the line must come out of this method as a `ReadingParseException`. Some problems you detect yourself, such as the wrong number of fields or an unknown station. Others are reported by Java as a different exception, which you must catch and translate into a `ReadingParseException`, keeping the original as the cause:

- `NumberFormatException` from `Double.parseDouble`;
- `DateTimeParseException` from `LocalDate.parse`;
- `IllegalArgumentException` from your `Reading` constructor.

5.5 Task 5: Reading the whole file

Add an instance method to `WeatherData`, `void loadReadings(Path file) throws IOException`, that reads every line after the header. Each good reading is kept. Each bad line is caught as a `ReadingParseException`, saved in a list, and logged, and the loop moves on: one bad line never stops the program. A file that cannot be read at all is the problem Task 1 handles, so `loadReadings` does not catch the `IOException`.

Logging. Use `java.util.logging`, in the one-line format we use in class. Log a `WARNING` for each bad line, and an `INFO` message when loading starts, when it finishes, and when the output files are written. For `readings.csv`, the console shows messages like these:

```
INFO: Loading files/readings.csv
WARNING: Skipped line 2: date is not a valid date: "2026-06-31"
WARNING: Skipped line 9: expected 5 fields but found 4
...
INFO: Loaded 43 readings and skipped 8 lines
```

Your wording may differ, but the levels may not.

The error log. Write the bad lines to `files/errors.txt`, following `sample_errors.txt`: the network's name, the file that was read, the number of lines skipped, and each exception's message, in file order.

6 Part 2: Ordering, Generics, and Collections (Week 13)

6.1 Task 6: Putting readings in order

Make `Reading` implement `Comparable<Reading>`. Readings are ordered by date, earliest first, and readings on the same date are ordered by station ID. After `loadReadings` has read the file, each station's readings must be in this order.

6.2 Task 7: A generic class

Write a generic class `MinMax` that keeps track of the smallest and largest of the values added to it. Use exactly this header:

```
public class MinMax<T extends Comparable<? super T>>
```

Table 2: The methods of `MinMax`.

Method	What it does
`void add(T value)`	Adds a value. A `null` value throws `NullPointerException`.
`T getMin()`	Returns the smallest value added. Throws `IllegalStateException` if none has been added.
`T getMax()`	Returns the largest value added. Throws `IllegalStateException` if none has been added.
`int getCount()`	Returns how many values have been added.
`boolean isEmpty()`	Returns `true` if no values have been added.

The report uses `MinMax<LocalDate>` for the first and last dates of the good readings, and `MinMax<Double>` for each station's temperature range, adding both its highs and its lows.

6.3 Task 8: Choosing the collections

`WeatherData` holds three collections. Declare each with the interface type shown, and choose the implementation yourself:

Table 3: The collections in `WeatherData`.

Collection	What it must do
`Map<String, Station>`	The stations, by ID. The report lists stations in ID order.
`Map<String, List<Reading»`	Each station's good readings, by station ID, also in ID order. A station with no good readings still has an entry, with an empty list.
`List<ReadingParseException>`	One exception for each bad line, in file order.

Also add these to `WeatherData`:

- a static method that reads `stations.csv`:

```
public static Map<String, Station> loadStations(Path file)
        throws IOException
```

- a constructor, `WeatherData(Map<String, Station> stations)`;
- methods that return a station's readings and the list of errors, named and written as you see fit.

Your code may not use raw types anywhere: write `List<Reading>`, never just `List`.

7 Part 3: Statistics with Streams (Week 14)

7.1 Task 9: The `WeatherStats` class

Write a class `WeatherStats` with the static methods in Table 4. Each takes a `Collection<Reading>` named `readings`, so it works on one station's list or on every reading at once. Two of them take one more parameter: `hottestDays` takes `int n`, and `daysAtOrAbove` takes `double threshold`. Write the body of each method as a single stream pipeline, with lambdas or method references and no loops.

Table 4: The methods of `WeatherStats`.

Method	Return type	Returns
`averageHigh`	`double`	The average high, or 0 if there are no readings
`averageLow`	`double`	The average low, or 0 if there are no readings
`totalPrecipitation`	`double`	The total precipitation
`rainyDays`	`long`	The number of readings with any precipitation
`hottest`	`Optional<Reading>`	The reading with the highest high, or an empty `Optional`
`hottestDays`	`List<Reading>`	Up to `n` readings, highest high first
`daysAtOrAbove`	`Map<String, Long>`	For each station, the number of readings with a high of `threshold` or more
`precipitationByStation`	`Map<String, Double>`	Each station's total precipitation

Two readings can have the same high. When they do, the one that comes first in natural order, from Task 6, counts as hotter. Define this order, hottest first with ties broken by natural order, once, as a `Comparator<Reading>`, and use it in both `hottest` and `hottestDays`. The two methods that return a map must keep the station IDs in order.

7.2 Task 10: The weather report

Finish `WeatherReport`. Its `main` method reads `files/stations.csv` and `files/readings.csv`, then writes `files/weather_report.txt` and `files/errors.txt`. It no longer prints the readings; the console shows only the log. The report follows `sample_weather_report.txt`, with these sections:

1. **A heading** with the first and last dates of the good readings, the number of good readings, and the number of lines skipped.
2. **One section per station,** in ID order: each reading in date order, the average high and low and the total precipitation, the number of readings and of rainy days, the temperature range, and the hottest day. A station with no good readings says `No readings`.
3. **Hottest Days**, the five hottest readings in the network.
4. **Days at or above 100 degrees**, for every station in ID order, including those with none.
5. **The wettest station**, the one with the most total precipitation.

Every average, total, range, and ranking in the report must come from a `WeatherStats` method or a `MinMax`. Loops are fine for printing. Match the sample closely: the same sections, in the same order, with the same information. Exact spacing may vary within reason, but the report must stay clean and aligned for both readings files.

8 Additional Sample Data

The file `readings2.csv` covers August, and it is used with the same `stations.csv`. Its bad lines are different from those in `readings.csv`, and two of them are tricky. One is a blank line. The other is a value that `Double.parseDouble` accepts as a number even though it is not one; if a total in your report looks strange, look closely at the lines your program kept. One station also sent nothing at all in August, which tests the `No readings` case and every method's handling of an empty list.

To test with it, change the readings file name your program reads and run it again. The program you submit should read `readings.csv`. Also change a file name to one that does not exist, to check that your program prints one clear message instead of a stack trace.

9 Requirements Checklist

- Your program grows out of the provided starter, and the classes and methods are named, and have the parameter and return types, that the tasks give.
- A file that cannot be read produces one clear message, not a stack trace.
- `Reading` is a record that refuses every value the network does not accept.
- `ReadingParseException` is checked and has both constructors, and `parseReading` reports every problem as one, keeping the cause when it translates another exception.
- One bad line never stops the program. Each one is logged as a warning and written to the error log.
- `Reading` implements `Comparable<Reading>`, and each station's readings are in natural order.
- `MinMax` works for any comparable type and handles the empty case, and the report uses it for both dates and temperatures.
- The collections in `WeatherData` keep the order the report needs, and there are no raw types.
- Every `WeatherStats` method is a single stream pipeline, and the hottest-first order is one `Comparator` used in both places.
- Your program writes the weather report and the error log, closely following the sample files, and both stay clean and aligned for both readings files. The console shows only the log.
- Your code meets the course's standards for code quality, and every file has the required header comment.

10 Grading

Items in Tables 5 and 6 are worth 1 point each and are either met or not met. Items in Table 7 are worth a range of points, with partial credit based on how well your work meets the standards we

set in lecture.

Table 5: Submission and setup (1 point each)

Item	Points
Submitted as a reasonable attempt that compiles and runs in Eclipse without errors	1
Project laid out as described in Project Setup (project name, packages, and `files` folder)	1

Table 6: Code quality (1 point each)

Item	Points
Consistent naming conventions used throughout	1
Consistent formatting and indentation throughout	1
No unrequested extra output	1
Each file has the required header comment	1
Loop and method logic reads clearly from top to bottom	1

Table 7: Implementation (partial credit possible)

Item	Points
Task 1: a file that cannot be read gives one clear message, with no stack trace	2
Task 2: `Reading` refuses every bad value, with clear messages	6
Tasks 3 and 4: the exception class, and `parseReading` translating every problem while keeping the cause	8
Task 5: loading continues past bad lines, and the error log is complete	6
Task 5: bad lines and milestones are logged at the right levels	2
Task 6: `compareTo` orders readings correctly, and each station's readings are sorted	6
Task 7: `MinMax` works for any comparable type and handles the empty case	6
Task 8: the collections keep the required order, with no raw types	4
Task 9: every `WeatherStats` method is a correct stream pipeline, with one comparator for the hottest-first order	10
Task 10: the report is complete and follows the sample for both readings files	7

Total: 64 points.

11 Submission

Submit through your course GitHub repository, per the syllabus's submission guidelines. Include your `src` folder with the `weather` and `app` packages, and the `files` folder with the input files. It is fine either way whether your output files are included, since your program creates them every time it runs.